University of Basel

Bare-Metal OS for a Raspberry Pi4

Final Presentation

Andrea Seehuber & Luca Fässler

Operating Systems Lecture Spring Semester 2026

June 2026

Outline

1. Problem Statement / Motivation
2. Hardware and Software Setup
3. System Architecture
4. Live Demo
5. Evaluation and Limitations
6. Lessons Learned and Conclusion

Problem Statement / Motivation

- **Goal:** Build a small OS kernel from scratch on real hardware
- No Linux, no standard C runtime, no user space
- Show core OS concepts in a concrete embedded system
- **Main challenge:** combine scheduling, hardware access, I/O, and diagnostics

Hardware and Software Setup

- **Raspbery Pi 4**, BCM2711, ARM Cortex-A72
- **Sense HAT** with:
  - 8x8 RGB LED Matrix
  - Joystick
  - Pressure, Humidity, Temperature Sensors
- **UART** as Control/Debug interface
- **HDMI** as visual output
- C & AArch64 assembly
- Built with Makefile and AArch64 bare-metal cross compiler
- **Output:** kernel8.img

System Architecture

MMIO Layer:

- Direct access to peripheral registers

Drivers:

- UART, GPIO, I2C, timer, interrupts, HDMI framebuffer

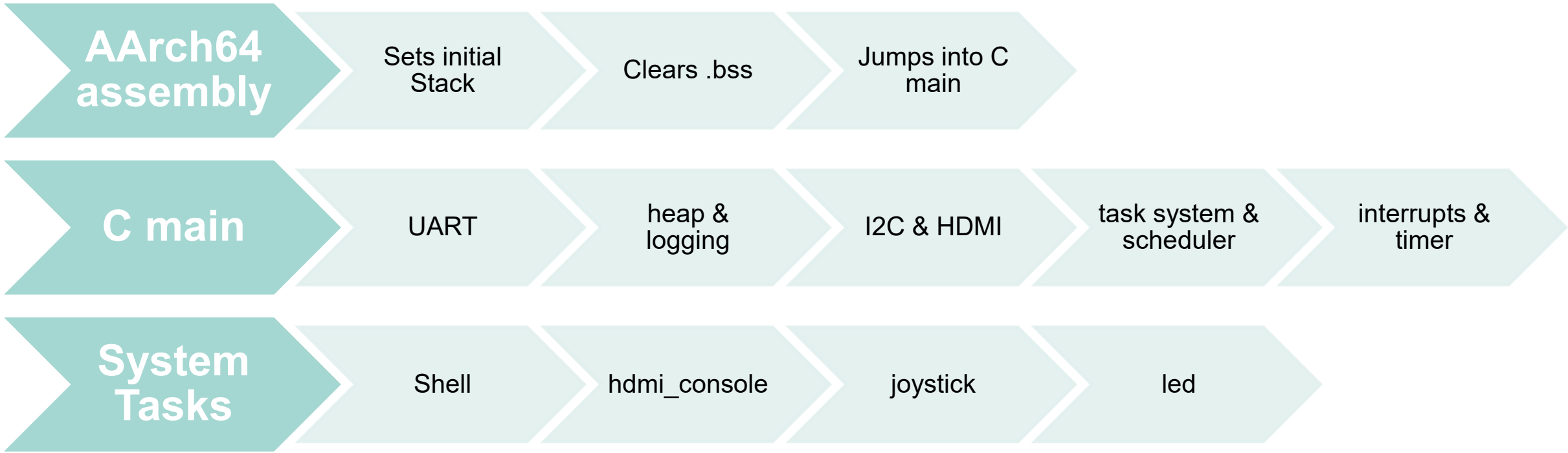
Kernel Mechanisms:

- Task system, scheduler, mutexes, wait queues, heap, logging, tracing, diagnostics

Kernel Tasks:

- System tasks and user-controllable tasks

Boot and Initialization



Task System and Cooperative Scheduler

- Static **task table**
- **Each task has:** ID, state, flags, stack pointer, name, private stack
- **Cooperative Scheduling:** Tasks must yield, sleep, block or exit
- **Round-Robin** selection of ready tasks
- **Idle task** runs if no task is ready
- **Task states:**
  - READY, RUNNING, SLEEPING, BLOCKED, DYING

```
find next READY task
switch context
if none ready: run idle task
```

Live Demo

Evaluation and Limitations

- **Achieved:**
  - Custom Kernel boots on Pi4
  - Cooperative scheduler works
  - UART shell and joystick menu work
  - HDMI output work
  - Sense HAT LED and sensors integrated
  - Tasks can be started/stopped interactively
  - Runtime diagnostics implemented

- **Limitations:**
  - I2C still timing-sensitive
  - HDMI rendering is complex and error-prone
  - Cooperative scheduler depends on well-behaved tasks
  - No user/kernel separation
  - No virtual memory
  - No filesystem
  - No memory protection

Lessons Learned and Conclusion

- **Lessons learned:**
  - Bare-metal development requires careful initialization order
  - Hardware documentation is essential
  - Debugging without OS support is slow but educational
  - Cooperative scheduling is simple, but shifts responsibility to tasks
  - Shared Hardware needs clear ownership
  - Diagnostics are not optional; they are necessary for debugging

- **Conclusion:**
  - Built a compact embedded OS kernel
  - Demonstrates real OS mechanisms on real hardware
  - Good foundation for future improvements:
    - better I2C recovery
    - preemptive scheduling
    - stronger memory/stack checks
    - persistent storage/logging